Runtime Analysis Practice

For each problem, determine the tightest Big-O runtime in terms of n.

Nested Loops

1) Evaluate each for loop separately
2) If nested, multiply

1

```
void firstLoopPractice(int n) {
    for (int i = 1; i < n * n; i++) {
        for (int j = 0; j < i; j++) {
            for (int k = 1; k < n; k = 2 * k) {
                work();
            }
        }
    }
}
```

$O(n^2) > n^2 * n^2 = O(n^4)$

$O(\log_2 n)$

$n^4 \cdot \log n = O(n^4 \log n)$

[1 2 4 8 ... 16 ... 32]

2

```
void secondLoopPractice(int n) {
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            for (int k = n; k > 1; k = k / 3) {
                work();
            }
        }
    }

    for (int m = 0; m < n * n * n; m++) {
        work();
    }
}
```

$O(n) * O(n) * O(\log_3 n) = O(n^2 \cdot \log n)$

$O(n^3)$

$+$

3

Assume that a binary min-heap currently stores `m` items, where `m <= n`. The `push` and `pop` operations each run in `O(log m)` time.

```
void heapPractice(vector<int>& values) {
    priority_queue<int, vector<int>, greater<int>> minHeap;

    for (int i = 0; i < values.size(); i++) {
        minHeap.push(values[i]);
    }

    while (!minHeap.empty()) {
        minHeap.pop();
    }
}
```

n
O(n)
O(log n) * O(n log n)
O(n) * O(log n) * O(n log n) / + = O(n log n)

Recursive Runtime Analysis

Use the Master Theorem when it applies. Determine the tightest Big-O runtime in terms of `n`.

4

```
void recursivePracticeA(int n) {
    if (n < 10) {
        work();
        return;
    }

    recursivePracticeA(n / 3);

    for (int i = 0; i < n * n; i++) {
        work();
    }
}
```

1) Write recurrence equation
2) Identify a, b, d
3) Plug in to $\frac{a}{b^d}$ & comp. to 1

$\frac{a}{b^d} < 1 \rightarrow \Theta(n^d)$

$\frac{a}{b^d} = 1 \rightarrow \Theta(n^d \log n)$

$\frac{a}{b^d} > 1 \rightarrow \Theta(n^{\log_b a})$

$T(n) = 1T\left(\frac{n}{3}\right) + n^2 \cdot c$ (a = 1, b = 3, d = 2)

$\frac{1}{3^2} < 1$

$\Theta(n^2)$

5

$T(n) = 2T(\frac{n}{2}) + n \cdot c$ (a = 2, b = 2, d = 1)

$\frac{2}{2^1} = 1$

```
void recursivePracticeB(int n) {
    if (n < 10) {
        work();
        return;
    }

    for (int i = 0; i < n; i++) {
        work();
    }

    recursivePracticeB(n / 2);
    recursivePracticeB(n / 2);
}
```

$O(n)$

$\Theta(n \cdot \log n)$

6

$T(n) = 6T(\frac{n}{2}) + n^2 \cdot c$ (a = 6, b = 2, d = 2)

$\frac{6}{2^2} > 1$

```
void recursivePracticeC(int n) {
    if (n < 10) {
        work();
        return;
    }

    for (int i = 0; i < n * n; i++) {
        work();
    }

    for (int call = 0; call < 6; call++) {
        recursivePracticeC(n / 2);
    }
}
```

$O(n^2)$

$\Theta(n^{\log_2 6})$